# Technical Paper

The Innovation Game

**Date** November 2, 2026
**Url** https://tig.foundation/

# Contents


1 **Optimisable Proof of Work (OPoW)** 2
Protocol details for Benchmarker's Proof of Work

1.1 Challenge factors . . . 2
1.2 Deposit factors . . . 3
1.3 Factor weights . . . 3
1.4 Influence . . . 3
1.5 Adoption . . . 5

2 **Rewards** 6
Reward schemes payable to the players of TIG

2.1 Reward scheme . . . 6
2.2 Benchmarker rewards . . . 7
2.3 Delegator rewards . . . 7
2.4 Code rewards . . . 8
2.5 Advance rewards . . . 8
2.6 Challange Owner Rewards . . . 9

A **Imbalance penalty** 10

A.1 Relative variance . . . 10
A.2 Imbalance penalty . . . 11
A.3 Imbalance penalty and market domination . . . 12
A.3.1 Benchmarker's influence gradient . . . 12
A.3.2 Market domination with uncapped qualification . . . 16

# 1 Optimisable Proof of Work (OPoW)

In this section, we outline the mathematical formulas used to compute the rewards and penalties assigned to a Benchmarker $i$ as a result of participation in benchmarking. Rewards are assigned to Benchmarker on a *per block* basis, and are computed based on the overall *influence* Benchmarker $i$ exhibits in the reward pool $\mathcal{P}_B$ that is attributed to the Benchmarkers. A Benchmarkers influence is composed two main factors: the *challenge factors* and the *deposit factors*.

## 1.1 Challenge factors

Let $f_x^i$ be the number of *qualifying* solutions submitted by Benchmarker $i$ for a challenge $x$, such that $x \in X$ where $X$ is the set of all challenges currently available. For each challenge $x$, there is a set of *challenge tracks* $\tau \in T(x)$, which can vary between challenges. The number of qualifying solutions in a given challenge $f_x^i$ is related to the per track count by a simple summation:

$$f_x^i = \sum_{\tau \in T(x)} f_x^i(\tau) \tag{1.1}$$

where $f_x^i(\tau)$ is the number of qualifying solutions for benchmarker $i$ in a challenge $x$ on track $\tau$. Benchmarkers are incentivized to use 'legacy' codes over new submissions which offer only very minor improvements. We define the *legacy multiplier* $\mathcal{L}_x(\tau)$ as the a multiplier assigned to each qualifying bundle on each Challenge track, and it is given by

$$\mathcal{L}_x(\tau) \in \left[1.0 - \frac{S_\mathcal{L}}{2}, 1 + \frac{S_\mathcal{L}}{2}\right] \tag{1.2}$$

where $S_\mathcal{L}$ is the span of the legacy multiplier. The current configuration sets $S_\mathcal{L} = 0.05$, resulting in a range $[0.975, 1.025]$. The legacy multiplier is linearly mapped in this configured range based on the code submission age, in a way that penalizes the newer code submissions and rewards older ones.

Suppose that there are a total of 3 qualifying bundles $b_1, b_2, b_3$ on a particular challenge track. Each bundle is assigned a legacy multiplier a unique value in the range, calculated in regular intervals. For example, if the age of the algorithm used to produce the benchmark $b_i$ is $A(b_i)$, and $A(b_1) > A(b_2) > A(b_3)$, then $b_1$ is assigned the value of 1.025, $b_2$ takes 1.0 and $b_3$ the smallest value of 0.975.

Different benchmarks will usually share the algorithms used to construct them. In this case, we assign the legacy multipliers as above, within groups the multipliers are assigned randomly block. For example, if $A(b_1) > A(b_2) = A(b_3)$, then 0.975 and 1.0 are assigned randomly to $b_2$ and $b_3$ on a per block basis.

The average legacy multiplier for Benchmarker $i$ on Challenge $x$ and Challenge track $\tau$ is $\bar{\mathcal{L}}_x^i(\tau)$, and is calculated using a simple mean

$$\bar{\mathcal{L}}_x^i(\tau) = \frac{1}{f_x^i} \sum_{j=1}^{f_x^i} \mathcal{L}_x^j(\tau). \tag{1.3}$$

For a total of $N_m$, the challenge factor for benchmarker $i$ on challenge $x$ is given by

$$\hat{f}_x^i = \frac{\sum_{\tau \in T(x)} f_x^i(\tau) \bar{\mathcal{L}}_x(\tau)}{\sum_{i=1}^{N_m} \sum_{\tau \in T(x)} f_x^i(\tau)}. \tag{1.4}$$

The age of a code submission is simply the difference between the current round and the round in which the code was submitted. The legacy multiplier incentivizes code innovators to submit codes which offer a relatively significant improvement to the overall state of the art, and newer codes which only offer none or marginal improvements in the quality would not be favorable for Benchmarkers to older,'legacy' codes.

## 1.2 Deposit factors

Alongside the challenge factors, a Benchmarker can earn rewards based on *self-deposits* and *delegations*.

The self-deposit $d^i_{self}$ is the amount of TIG that a Benchmarker $i$ has deposited. The delegated amount $d^i_{del}$ is the amount of TIG that have been delegated to the Benchmarker $i$ by other token holders. Both of these quantities contribute to the overall reward payout. The corresponding self-deposit factor $D^i_{self}$ and delegated deposit factors are determined by

$$D^i_{self} = \min\left\{\frac{1.25}{|X|}\sum_{x\in X}\hat{f}^i_x, \frac{d^i_{self}}{\sum_{j=1}^{N_m} d^j_{self}}\right\} \tag{1.5}$$

$$D^i_{del} = \min\left\{\frac{1.25}{|X|}\sum_{x\in X}\hat{f}^i_x, \frac{d^i_{del}}{\sum_{j=1}^{N_m} d^j_{del}}\right\} \tag{1.6}$$

Each of these deposits measure Benchmarker $i$'s share of the corresponding total deposits, capped in proportion to the it's average share of qualifying solutions across Challenges. The variable challenge factor weighting allows us to onboard new challenges smoothly into the protocol.

## 1.3 Factor weights

The challenge factors and deposit factors both contribute to the influence of the benchmarker in the reward pool with an unequal weighting. Under the current protocol configurations, the challenge factors are responsible for 75% of the contribution towards a Benchmarkers influence, while the other 25% is accounted for by the two deposit factors.

Given a total number of $n$ Challenges, the challenge factor weights $\{w^f_j\}_{j=1}^n$ are fixed for all challenges and are determined simply by

$$w^f_j = \frac{0.75}{n} \tag{1.7}$$

such that $\sum_{j=1}^n w^f_j = 0.75$. The self-deposit $w^D_{self}$ and delegated $w^D_{del}$ factor weights are split evenly, with both factors taking an equal weight of $w^D_{self} = w^D_{del} = 0.125$. The sum of all factors weights is equal to 1.

## 1.4 Influence

Let $\mathcal{F}^i$ denote the set of all factors $\{f^i_1, \cdots, f^i_n, D^i_{self}, D^i_{del}\}$ for a Benchmarker $i$ and a corresponding weight set $\mathcal{W} = \{w^f_1, \cdots, w^f_n, w^D_{self}, w^D_{del}\}$ for each of the entries in $\mathcal{F}^i$. The size of $\mathcal{F}^i$ is then given by the sum of all challenges and the two deposit factors, i.e $|\mathcal{F}| = n + 2$.

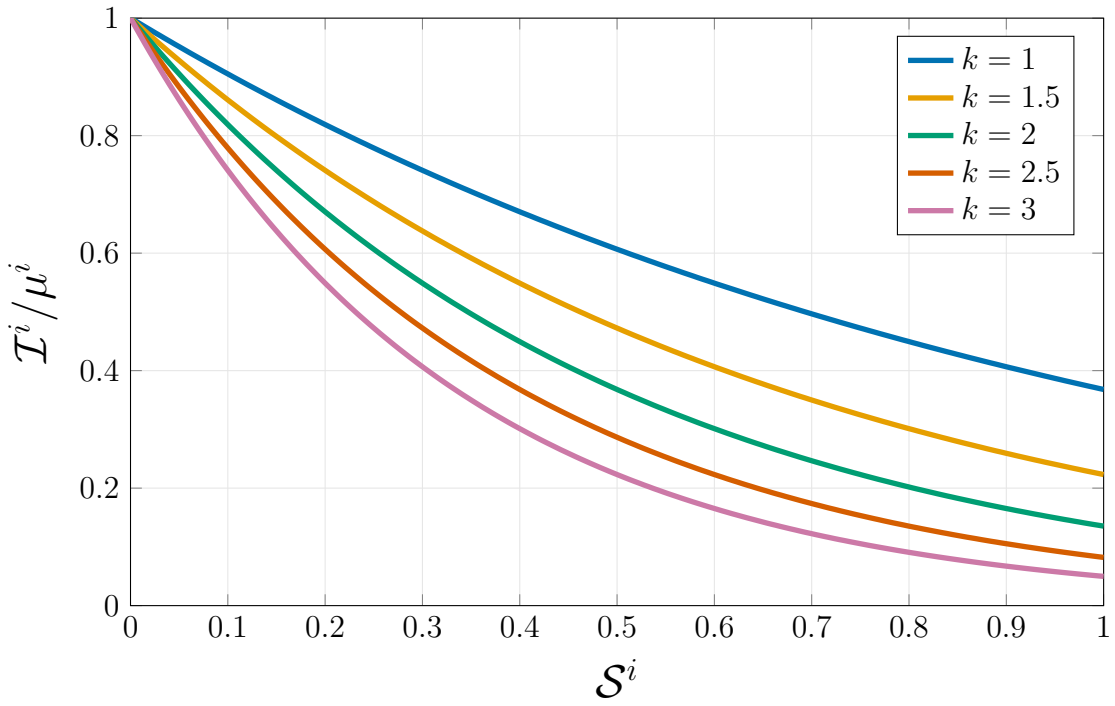



Figure 1: The effect of the security parameter $k$ on the imbalance penalty $\mathcal{I}^i/\mu^i = \exp(-k\mathcal{S}_i)$ for a Benchmarker $i$.

We now introduce the mean $\mu^i$ and the variance $(\sigma^i)^2$ of $\mathcal{F}^i$, which is given by the standard weighted mean and variance formulae

$$\mu^i = \frac{\sum_{j=1}^{n+2} w_j f_j}{\sum_{j=1}^{n+2} w_j} \tag{1.8}$$

$$(\sigma^i)^2 = \frac{\sum_{j=1}^{n+2} w_j (f_i - \mu^i)^2}{\sum_{j=1}^{n+2} w_j} \tag{1.9}$$

where $f_j \in \mathcal{F}^i$ and $w_j \in \mathcal{W}$. The equality of all factors $f_j$ is called a *parity*. Optimisable Proof of Work binds Challenges through a *parity mechanism* that rewards the Benchmarkers that balance their participation across all challenges, self-deposits and tokens delegated from other token holders. We define the factor imbalance as

$$\mathcal{S}^i = \frac{(\sigma^i)^2}{\mu^i(1-\mu^i)} \tag{1.10}$$

The factor imbalance measures the spread of the factors relative to the largest possible variance at that mean. For factors which are similar, $\mathcal{S}_i$ will be small, while a large variation between factors will result in a higher imbalance value $\mathcal{S}_i$. A Benchmarker $i$'s *un-normalised influence* is then determined by the mean of the factors, penalised by the imbalance

$$\mathcal{I}^i = \mu^i \exp(-k\mathcal{S}^i) \tag{1.11}$$

where the parameter $k$ is a fixed 'secruity' constant that is set by the protocol. This parameter $k$ sets the trade between the penalisation of honest specialization and bounding the influence obtainable from a concentrated advantage. The current choice for $k$ is set to $k = 2$.

By construction, the imbalance term $\mathcal{S}^i$ is necessarily bounded in the range $[0, 1]$ (see Appendix A.1). For perfect parity, we have that $S^i = 0$ and no penalty applies to the Benchmarker. The maximal penalty for a Benchmarker lies on the other extreme where $S^i = 1$,

which corresponds to a penalty of $\exp(-k)$, which corresponds to approximately 86% reduction of influence. The security parameter $k$ is an arbitrary choice, and can be adapted throughout the lifecycle of the protocol in order to ensure fairness, and incentivize the desire for parity (see Fig. 1. Increasing the value of $k$ applies stricter penalties on the Benchmarkers influence (see Appendix. A.2).

Normalising across all $N_m$ Benchmarkers gives

$$\mathcal{J}^i = \frac{\mathcal{I}^i}{\sum_{j=1}^{N_m} \mathcal{I}^j} \tag{1.12}$$

The normalised influence $\mathcal{J}^i$ determines the Benchmarker's expected reward share in the overall Benchmarker reward pool, quantifying the contribution of the Benchmarker across all challenges. Under a fully decentralized protocol, $\mathcal{J}^i$ also represents the probability that Benchmarker $i$ would produce the next block in the blockchain.

## 1.5 Adoption

The *adoption* of a code submissions quantifies the overall usage of the code by benchmarkers for a particular challenge.

Benchmarkers are incentivised to use code submissions which maximize their average bundle quality per track. As a result, code submissions which on average result in the highest bundle qualities will maximize the profits of both the Benchmarkers (more qualifying bundles scored by the frontier mechanism) and innovators (higher adoption rate which is directly proportional to the innovator rewards in the code submission reward pool).

The adoption $A_k$ of a code submission $k \in \mathcal{K}$, where $\mathcal{K}$ is the set of all code submissions for some given challenge and block , is given by

$$A_k = \sum_{i=1}^{N_m} \left( \frac{f_k^i}{\sum_{k' \in \mathcal{K}} f_{k'}^i} \right) \mathcal{J}^i \tag{1.13}$$

where similarily to Sec. 1.1 we have defined $f_k^i$ as the number of qualifying bundles that a Benchmarker $i$ produced with Code submission $k$.

As Advance submissions offer a significant improvement to the current state of the art, subsequent code submissions which attempt to improve upon the novel idea introduced by the advance should not be eclipsed by minor improvements. The adoption of an Advance submission $m$ is the sum of all of the adoptions of the Code submissions which reference $m$. Let $K_m$ be the subset $K_m \subset \mathcal{K}$ of code submissions which reference the advance submission $m$. The adoption $A^m$ of advance $m$ is determined by

$$A^m = \sum_{k \in K_m} A_k \tag{1.14}$$

Code submissions reference the Advance submissions Unique Algorithm Identifier (UAI). There is no automatic penalty applied to Code submissions which improve upon the novelty of the advance submission without referencing it. The innovator behind the advance submission is incentivized to report such algorithms to TIG by an unfair loss of tokens through a reduced adoption. Reported code submissions are checked internally and banned.

# 2 Rewards

Rewards are payable to players based on the current TIG token emission schedule, shown in Table. 1.

| Tranche | Rounds | Emissions per block | Emissions per round | Start date | End date |
|---|---|---|---|---|---|
| 1 | 1 - 26 (26) | 100 TIG | 1,008,000 TIG | 24 Nov 2023 | 1 June 2024 |
| 2 | 27 - 78 (52) | 50 TIG | 504,000 TIG | 1 June 2024 | 15 Aug 2025* |
| 3 | 79 - 182 (104) | 25 TIG | 252,000 TIG | 15 Aug 2025* | 13 Aug 2027* |
| 4 | 183 - 390 (208) | 12.5 | 126,000 TIG | 13 Aug 2027* | 8 Aug 2031* |
| 5 | 391 - 806 (416) | 6.25 | 63,000 TIG | 8 Aug 2031* | 29 July 2039* |

** Approximates*

Table 1: TIG Token emission schedule.

Not all of the tokens which are emitted per block are distributed amongst the players, the remaining portion of tokens are temporarily stored within the *TIG vault*. The proportion of TIG tokens payable to the players is directy affected by the number of active Challenges in the protocol. Throughout the lifetime of the protocol, the network is still growing - as new challenges are added the proportion of tokens rewarded to players will increase to an asymptotic value.

Concretely, we define a scaling function $\Gamma : \mathbb{N} \to [1, 1.02]$ that determines the proportion of emissions to be distributed (with the rest sent to the vault) is based on the current number of active Challenges $n$ such that

$$\Gamma(n) = 1.0258(1 - 0.8730e^{-0.0354n}) \tag{2.1}$$

As the network approaches a total of 100 active Challenges, $\Gamma(n) \to 1$ and the rewards payable to the players will coincide with the original token emission schedule. As the network grows past 100 active challenges, $\Gamma(n)$ will grow asymptotically towards a value of 1.02, and the TIG tokens which were previously stored in the vault are released at a rate of around $2\%$ of the current block token emission rate.

## 2.1 Reward scheme

Under the current scheme, player reward distribution is performance based and distributed in the following way:

- **Benchmarkers - 50%**: Distributed amongst Benchmarkers and delegators pro-rata based on each Benchmarker's influence (see Secs. 2.2 and 2.3)
- **Innovators (Advances) - 20%**: Advance submissions will earn 20% of the block rewards, which are split evenly across challenges, before being distributed to Innovators,

pro-rata with each of the Advance's submission*.

- **Innovators (Code) - 10%**: These rewards are split evenly across Challenges, before being distributed to Innovators, pro-rata with each of their Code's adoption*.
- **Challenge Owners (Base) - 10%**: These rewards are split evenly across all Challenges*.
- **Challenge Owners (Performance) - 10%**: There is a secondary reward for Challenge Owners distributed pro-rate with the number of active Advance submissions that have recently been submitted to the Challenge*.

The protocol offers a substantially higher reward payout to Challenge Owners and Innovators for advance submissions to incentivize the submission of algorithms that offer signficant improvements to the current state-of-the-art. Such innovations are of typically substantial commercial value and are strongly encouraged through the reward scheme.

## 2.2 Benchmarker rewards

Benchmarkers receive awards from the Benchmarkers reward pool $\mathcal{P}_B$, such that

$$\mathcal{P}_B = R_{block} \cdot r_B \tag{2.2}$$

where $R_{block}$ is the total reward for the block and $r_B$ is the percentage of the block rewards which are shared among all of the Benchmarkers. Currently, $r_B$ is set to 0.5, which assigns 50% of the block rewards to the Benchmarkers.

A Benchmarker $i$ then receives a reward $R_B^i$ based on the total Benchmarker reward pool and their individual influence by

$$R_B^i = \mathcal{P}_B \cdot \mathcal{J}^i \tag{2.3}$$

## 2.3 Delegator rewards

Benchmarkers receive awards based on the amount of benchmarking they do, as well as delegated deposits from other token holders.

Delegators are rewarded for their delegations to Benchmarkers by receiving a small share of the Benchmarkers rewards to which they delegated to.

For a Benchmarker $i$, a delegator $d \in \mathcal{D}$ where $\mathcal{D}$ is the set of all delegators receives a reward $R_d^i$ based on

$$R_d^i = R_B^i \cdot d_s \cdot \frac{d_d^i}{\sum_{d' \in \mathcal{D}} d_{d'}^i} \tag{2.4}$$

where $d_s$ represents the portion of rewards that a Benchmarker $i$ shares to its delegators (set between 0 and 0.25) and $d_d^i$ represents the amount of TIG tokens which are delegated to a benchmarker $i$ by $d$. By default, the delegator share $d_s$ is set to be 0.15 (15%) and can be changed on an individual basis by the Benchmarker. The reward calculation is simply the fraction of the Benchmarker rewards that are shared with the delegators multiplied by the delegator $d$'s contribution to the delegated amount. The summation in the last term of Eqn. 2.4 is nothing more than the delegated amount $d_{self}^i$ in Sec. 1.2.

*In certain cases, rewards may be "orphaned", where code/advance submissions may not meet the adoption threshold; or a Challenge may not have an owner. These rewards are redirected into a "bootstrap" address in the TIG Treasury, specifically ringfenced to incentivize algorithmic breakthroughs

The total reward a delegator $d$ receives from delegating to Benchmarkers is then calculated as the sum

$$R_d = \sum_{i=1}^{N_m} R_d^i \tag{2.5}$$

## 2.4 Code rewards

The Code reward pool $\mathcal{P}_C$ constitutes 10% of the total reward, which is then split equally across all challenges.

Code rewards are calculated based on two different *states*:

- Active state: A code submission will become active 4 rounds after it has been submitted.
- Merged state: A code submission is merged into the `main` repository branch if it accumulates enough merge points in a singular round (see Sec.???)

For an active code submission $k \in \mathcal{K}$, the code submission reward $R_k$ is determined based on the overall adoption:

$$R_k = \begin{cases} \frac{\mathcal{P}_C}{n} A_k & A_k \geq 0.25 \\ 0 & A_k < 0.25 \end{cases} \tag{2.6}$$

where $A_k$ is the adoption and $n$ is the total number of active challenges. Code submissions which are in the active state will only be eligible for block rewards if their respective adoption is above $25\%$. For algorithms which are in the merged state the adoption threshold is dropped, and the code reward is

$$R_k = \frac{\mathcal{P}_C}{n} A_k \tag{2.7}$$

Algorithms which are in the active state but do not exceed the adoption threshold will have their rewards redirected into the TIG bootstrap fund. The total reward that ends up in the TIG bootstrap is simply calculated by

$$\frac{\mathcal{P}_C}{n} - \sum_{k \in \mathcal{K}} R_k$$

## 2.5 Advance rewards

The advance reward pool $\mathcal{P}_A$ constitutes a total of 20% of the block reward, which is then split evenly across all challenges. Reward calculations are calculated in much the same way as for the code submissions, with two key differences:

- For an advance submission $m$ which is in the active state, the adoption threshold is 50%.
- Advance submissions rewards are calculated based on all of the code submissions which reference $m$.

If $K_m \subset \mathcal{K}$ is the set of all code submissions which reference $m$, the advance reward $R^m$ is given by

$$R_m = \frac{\mathcal{P}_C}{n} A^m = \frac{\mathcal{P}_C}{n} \sum_{k \in K_m} A_k \tag{2.8}$$

If $K_m = \emptyset$, then the advance submission will not receive any awards. As in Sec. 2.4, uncaptured rewards are redirected into the TIG Boostrap fund. For a set $\mathcal{M}$ of all advance submissions (for a given challenge and block), the total amount of TIG that ends up in the TIG bootstrap fund is

$$\frac{\mathcal{P}_C}{n} - \sum_{m \in M} R_m$$

## 2.6 Challange Owner Rewards

Challenge owners play an integral part in the lifetime of challenges, by steering the direction and difficulty of Challenges to stay competitive with the state-of-the-art algorithms and ensuring alignment with practical application.

Rewards to challenge owners are payable from a pool $\mathcal{P}_{O_w}$:

- A static challenge owner reward pool: Block rewards is assigned as a fixed reward to all Challenge owners.
- A dynamic challenge owner reward pool: A portion of the the block rewards are also set aside as a performance-based reward for challenge owners which are distributed pro-rata based on the number of Recent Advances. If there are no Recent Advances, this reward is evenly distributed to all Challenge Owners.

Recent Advances are defined as advance submissions which have passed the Advance Rewards vote in the past 15 rounds. The reward $R_{O_w}$ a challenge owner $O_w$ receives is the sum of their static rewards $R^{stat}_{O_w}$ and dynamic rewards $R^{dyn}_{O_w}$ such that

$$R_{O_w} = R^{stat}_{O_w} + R^{dyn}_{O_w} \tag{2.9}$$

The static reward is computed simply as

$$R^{stat}_{O_w} = \frac{\mathcal{P}_{O_w}}{n} r_{stat} \tag{2.10}$$

where $r_{stat}$ is a parameter $r_{stat} \in [0, 1]$ set to regulate the proportion of the Challenge owner reward pool to assign as a static reward. Let $n^i_{adv}$ be the number of recent advances made by challenge owner $i$. If $\sum_j n^j_{adv} \neq 0$, then the dynamic reward is given by

$$R^{dyn}_{O_w} = \frac{n^i_{adv}}{\sum_j n^j_{adv}} \mathcal{P}_{O_w}(1 - r_{stat}) \tag{2.11}$$

If there are no Recent Advances, the dynamic reward for each Challenge owner $i$ is then

$$R^{dyn}_{O_w} = \frac{\mathcal{P}_{O_w}}{n}(1 - r_{stat}) \tag{2.12}$$

# References

[1] Rajendra Bhatia and Chandler Davis. A better bound on the variance. *The american mathematical monthly*, 107(4):353–357, 2000.

# A Imbalance penalty

## A.1 Relative variance

Given a set of factors $f_i \in [0, 1]$ with corresponding weights $w_i \geq 0$ such that $\sum_i w_i = 1$, the weighted mean and variance are determined by

$$\mu = \sum w_i f_i \tag{A.1}$$

$$\sigma^2 = \sum w_i (f_i - \mu)^2 \tag{A.2}$$

In Sec. 1.4, we defined the imbalance $S$ as the spread of factors

$$S = \frac{\sigma^2}{\mu(1-\mu)} \tag{A.3}$$

In this subsection we will motivate the use of the equation above, often referred to as the bounded variance, to measure the spread of factors.

Suppose that instead, we use the relative variance $S_{CV}$ to define the spread, such that

$$S_{CV} = \frac{CV^2}{n-1} \tag{A.4}$$

where $CV = \sigma/\mu$ is the coefficient of variation with $n-1$ degrees of freedom. The relationship between $S$ and the relative variance is given by

$$S = S_{CV} \frac{(n-1)\mu}{1-\mu} \tag{A.5}$$

The difference between the two equations becomes immediately clear when we consider a concrete example. Consider a set of 4 equally weighted factors, if we compute $S_{CV}$ and $S$ we find that Relative variance fails to capture the imbalance of the factors relative to the mean.

| Factors | $S_{CV}$ | $S$ |
|---|---|---|
| (0.04,0,0,0) | 1 | 0.0303 |
| (0,40,0,0,0) | 1 | 0.3333 |
| (1.00,0.0.0) | 1 | 1 |

Across all three sets, the factors would suffer a maximum penalty under relative variance, even though the parity is much better for the first set compared to the last. As a result, the calculation of $S$ makes it much easier for smaller participants who might have a smaller share of influence to be involved, as the penalty is balanced with respect to participation. Smaller participants suffer smaller consequences for specialisation while larger players are expected to diversify. The reason behind this is simple: a smaller participant may not have the sufficient compute to diversify across many challenges. Additionally, a small participant will not be able to manipulate the network share through specialisation and as a result does not carry a material risk of market manipulation.

Both $S_{CV}$ and $S$ lie within the $[0, 1]$ under equal weighting. The bounded-variance measure $S$ retains this bound even under unequal weights resulting from the fact that $\sigma^2 \leq \mu(1-\mu)$ is a tight bound [1].

## A.2 Imbalance penalty

The imbalance penalty is an important mechanic to ensure fairness, and to prevent the domination of a singular Benchmarker in one challenge. The key consideration that we address here is the choice of the security parameter $k$, which dictates the severity of the imbalance $S^i$ for a Benchmarker $i$.

Let's suppose that for a given block, we want to directly capture the effect of shifting the value of $k$ on a Benchmarker's influence. Let $Z(k) = \sum_j \mathcal{I}^j$ be the normalisation factor of a Benchmarker $i$'s influence, such that $\mathcal{J}^i = \mathcal{I}^i/Z(k)$ and suppose that $Z(k), \mu^i > 0$. Taking logarithms, we find that

$$\log \mathcal{J}^i = \log \mu^i - kS^i - \log Z(k) \tag{A.6}$$

and then differentiating with respect to parameter $k$ gives

$$\frac{d \log \mathcal{J}^i}{dk} = -\frac{Z'(k)}{Z(k)} - S^i \tag{A.7}$$

where $Z'(k)$ represents a derivative with respect to $k$. In fact, we can show that

$$Z'(k) = \frac{d}{dk}\left(\sum_j \mu^i e^{-kS^j}\right) = -\sum_j (\mu^j e^{-kS^j}) S^j = -\sum_j \mathcal{I}^j S^j = -Z(k) \sum_j \mathcal{J}^j S^j \tag{A.8}$$

We define the influence-weighted average imbalance $\bar{S}_{\mathcal{J}} = \sum_j \mathcal{J}^j S^j$ since $\mathcal{J}^i$ represent the normalized influence weights which sum to 1. Substituting, the relative effect of the Benchmarker $i$'s influence due to $k$ is given by an elegant equation

$$\frac{d \log \mathcal{J}^i}{dk} = \bar{S}_{\mathcal{J}} - S^i \tag{A.9}$$

From the result above, it is clear how a Benchmarker's influence is sensitive to a change in the value of $k$. For a Benchmarker with a high parity (and therefore a small $S^i$) will have $S^i \leq \bar{S}_{\mathcal{J}}$, and as a result an increase in $k$ should positively affect the Benchmarker's influence, proportional to the deviation from the mean. In the opposite direction: a Benchmarker that does not balance their factors well will be negatively affected by an increase in the value of $k$. If a Benchmarker $i$ has an imbalance penalty $S^i \approx \bar{S}_{\mathcal{J}}$, an instanteneous change in $k$ will not affect the Benchmarker's influence in any significant way. It is interesting to note that the imbalance penalty only remains effective as long as the distribution of the $S^i$ values has a sufficiently large variance. Ideally for the protocol, the distribution of the imbalance $S^i$ should be such that for each Benchmarker $i$, we minimize Eqn. A.9 and therefore minimize the impact of an imbalance penalty. However this on its own is an insufficient condition to ensure parity. If there are 4 Benchmarkers and 4 Challenges, and each Benchmarker focused on 1 challenge, $\bar{S}_{\mathcal{J}} = S^i$ for all $i$ and the imbalance penalty is ineffective, and there is no competition within the challenge to drive innovation.

It should be noted that in Eqn. A.9, the change in $\mathcal{J}^i$ is only a sensitivity guideline, and it does not imply that for a larger deviation in $k$ the same Benchmarker will continue to either gain or lose influence. The influence-weighted average imbalance $\bar{S}_{\mathcal{J}}$ will vary with $k$, and more importantly the equation does not take into account how the influence of one Benchmarker $i$ is affected by a change of factors by another Benchmarker $j$. In general, the instantaneous influence shift due to a change in the parameter $k$ is given by

$$\mathcal{J}^i(k + \Delta k) = \frac{\mathcal{J}^i(k) e^{-\Delta k S^i}}{\sum_j \mathcal{J}^j(k) e^{-\Delta k S_j}} \tag{A.10}$$

## A.3 Imbalance penalty and market domination

In an idealised TIG model, the amount of compute which is expended by the Benchmarkers results in an equal share of Challenge factors spread across the challenges for each Challenge. The security parameter $k$ sets a precedent on how much a Benchmark is penalised by being out of parity. If the value of $k$ is small, a Benchmarker $i$ may no longer be incentivized to return to parity, on the grounds that forcing compute into a particular challenge will be more profitable than balancing compute across different challenges. Without a sufficient driving factor behind penalisation, there is no incentive for a Benchmarker to interact with a particular challenge if the compute will not lead to a substantial increase in reward.

For example, consider a Benchmarker $i$ who does not have access to a GPU. For a sufficiently large $k$, the Benchmarker would incur a large penalty based on not being able to interact with the GPU-based challenges. It therefore may be more profitable for the Benchmarker to invest or rent a GPU machine to partake in the GPU-challenges and therefore earn much higher rewards.

There is another motivation behind having a sufficiently large value of $k$. Suppose that $k$ is very small; a Benchmarker $i$ may intentionally force themselves out of parity in order to gain rewards from another source. For example, if Benchmarker $i$ decides to submit an algorithm $k$ which may or may not offer a substantial increase to the average bundle quality, they could in theory attempt to funnel all of their compute into this one challenge, and thus earn the adoption rewards from Benchmarking their own algorithms.

The question remains, whether there is some incentive in order to cap the value of $k$ at some larger value. If the penalty is large enough, a large Benchmarker could drive an excessive fraction of the total cost into a specific challenge. Even though a Benchmarker is penalised more heavily through a deviation from parity, they can also push the other Benchmarkers out of parity by pushing more compute into one challenge. The outcome of this, is that it can become more favourable for a large Benchmarker to drive a higher compute disparity. In this section, we will show that the turning point at which a change in one of the challenge factors is no longer detrimental or beneficial, depending on the increment.

### A.3.1 Benchmarker's influence gradient

The derivative of the normalised Benchmarker $i$'s influence $\mathcal{J}^i$ with respect to the change in factor $f_x^j$ is given by

$$\frac{d\mathcal{J}^i}{df_x^j} = \sum_{a,b,q,y,z} \frac{\partial \mathcal{J}^i}{\partial \mathcal{I}^a} \frac{\partial \mathcal{I}^a}{\partial \hat{f}_z^b} \frac{\partial \hat{f}_z^b}{\partial f_y^q} \frac{df_y^q}{df_x^j} \tag{A.11}$$

subject to the condition that Benchmarker $j$'s total cost $C^j = \sum_z g_z^j f_z^j$ is held constant, where $g_z^j$ is the unit cost for factor $f_z^j$. We now suppose that for Benchmarker $j$, we adjust the value of $f$'s in such a way that the their total cost remains the same. This in practise is done by removing compute from one Challenge and inserting it into another. Therefore the total rate of change with respect to factor $f_x$ is given by

$$0 = \frac{d}{df_x} \sum_z g_z f_z = \sum_z g_z \frac{df_z}{df_x} = g_x + \sum_{z \neq x} g_z \frac{df_z}{df_x}, \tag{A.12}$$

where we have dropped the $j$ notation for the Benchmarker. Consider now that the Benchmarker increments $c_x = g_x f_x$ by incrementing $f_x$, and that the corresponding changes in $c_{z \neq x}$

are proportional. Then it follows that

$$\frac{dc_z}{dc_x} = \frac{d(g_z f_z)}{d(g_x f_x)} = -qc_z, \quad z \neq x \tag{A.13}$$

where $q$ is a constant of proportionality to be determined. From the above, we find that

$$g_z \frac{df_z}{df_x} = -qg_x g_z f_z, \tag{A.14}$$

and substituting into Eqn. A.12 gives the value of $q$:

$$q = \frac{1}{\sum_{z \neq x} g_z f_z}. \tag{A.15}$$

Therefore, we find that the derivative is determined by

$$\frac{df_z}{df_x} = -\frac{g_x f_z}{\sum_{v \neq x} g_v f_v}, \quad z \neq x \tag{A.16}$$

For a general Challenge $y$, the change of one factor with respect to another is given by

$$\frac{df_y}{df_x} = \frac{\partial f_y}{\partial f_x} + \sum_{v \neq x} \frac{\partial f_y}{\partial f_v} \frac{df_v}{df_x} \tag{A.17}$$

$$= \delta_{xy} + \sum_{v \neq x} \delta_{yv} \frac{df_v}{df_x} \tag{A.18}$$

$$= \delta_{xy} + \sum_{z} \bar{\delta}_{xv} \delta_{yv} \frac{\partial f_z}{\partial f_x} \tag{A.19}$$

where $\delta_{xy}$ is the Kronecker delta function, and $\bar{\delta}_{xy} = 1 - \delta_{xy}$. Simplifying, we can show that

$$\frac{df_y}{df_x} = \delta_{xy} - \bar{\delta}_{xy} \frac{g_x f_y}{\sum_{v \neq x} g_v f_v}. \tag{A.20}$$

We can now begin with computing the change in influence of Benchmarker $i$. Firstly, we split the derivative using one chain rule definition, which gives

$$\frac{d\mathcal{J}^i}{df_x^j} = \sum_{q,y} \frac{\partial \mathcal{J}^i}{\partial f_y^q} \frac{df_y^q}{df_x^j} = \sum_{q,y} \frac{\partial \mathcal{J} i}{\partial f_y^q} \left( \delta_{qj} \frac{df_y^j}{df_x^j} \right) \tag{A.21}$$

Directly subsituting Eqn. A.20 into the equation above and applying the Kronecker delta we can obtain

$$\frac{d\mathcal{J}^i}{df_x^j} = \sum_{y} \frac{\partial \mathcal{J}^i}{\partial f_y^j} \left( \delta_{xy} - \bar{\delta}_{xy} \frac{g_x^j f_y^j}{\sum_{v \neq x} g_v^j f_v^j} \right) \tag{A.22}$$

The remaining partial derivative can be further expanded by a second use of the chain rule in terms of the normalised challenge factors $\hat{f}_x^j$, such that

$$\frac{d\mathcal{J}^i}{df_x^j} = \sum_{a,y} \frac{\partial \mathcal{J}^i}{\partial \hat{f}_y^a} \frac{\partial \hat{f}_y^a}{\partial f_y^j} \left( \delta_{xy} - \bar{\delta}_{xy} \frac{g_x^j f_y^j}{\sum_{v \neq x} g_v^j f_v^j} \right) \tag{A.23}$$

To progress further we neglect the Legacy multiplier factor, which here we set to 1 for all tracks and challenges, and assume a uniform weighting across all challenge factors. As a result, the normalised challenge factors can be written in the simple form

$$\hat{f}_x^i = \frac{f_x^i}{\sum_j f_x^j}. \tag{A.24}$$

Now differentiating the above with respect to challenge factor $f_x^j$, we find that

$$\frac{\partial \hat{f}_x^i}{\partial f_x^j} = \frac{\delta_{ij}}{\sum_k f_x^k} + f_x^i \frac{\partial}{\partial f_x^j}\left(\frac{1}{\sum_k f_x^k}\right) = \frac{\delta_{ij}}{\sum_k f_x^k} - \frac{f_x^i / \sum_k f_x^k}{\sum_k f_x^k} = \frac{\delta_{ij} - \hat{f}_x^i}{\sum_k f_x^k}. \tag{A.25}$$

Now substituting the result above into Eqn. A.23 results in the following:

$$\frac{d\mathcal{J}^i}{df_x^j} = \sum_{a,y} \frac{\partial \mathcal{J}^i}{\partial \hat{f}_y^a} \frac{\delta_{aj} - \hat{f}_y^a}{\sum_k f_y^k} \left(\delta_{xy} - \bar{\delta}_{xy} \frac{g_x^j f_y^j}{\sum_{v \neq x} g_v^j f_v^j}\right) \tag{A.26}$$

We now make a number of subtle manipulations. Firstly, multiply the equation above by $\sum_k f_x^k$, to obtain

$$\frac{d\mathcal{J}^i}{df_x^j} = \sum_{a,y} \frac{\partial \mathcal{J}^i}{\partial \hat{f}_y^a} \frac{\delta_{aj} - \hat{f}_y^a}{\sum_k f_x^k} \left(\delta_{xy} - \bar{\delta}_{xy} \frac{g_x^j f_y^j}{\sum_{v \neq x} g_v^j f_v^j}\right) \frac{\sum_k f_x^k}{\sum_k f_y^k}. \tag{A.27}$$

From the definitions of the normalised challenge factors, we can show that

$$\frac{\sum_k f_x^k}{\sum_k f_y^k} = \frac{f_x^j / \hat{f}_x^j}{f_y^j / \hat{f}_y^j} = \frac{\hat{f}_y^j}{\hat{f}_x^j} \frac{f_x^j}{f_y^j} \tag{A.28}$$

Substituting the above into Eqn. A.27 gives

$$\frac{d\mathcal{J}^i}{df_x^j} = \sum_{a,y} \frac{\partial \mathcal{J}^i}{\partial \hat{f}_y^a} \frac{\delta_{aj} - \hat{f}_y^a}{\sum_k f_x^k} \left(\delta_{xy} - \bar{\delta}_{xy} \frac{g_x^j f_x^j}{\sum_{v \neq x} g_v^j f_v^j} \frac{\hat{f}_y^j}{\hat{f}_x^j}\right). \tag{A.29}$$

Note here that multiplying $\sum_k f_x^k / \sum_k f_y^k$ factor by the Kronecker delta simply leads to the cancellation of the factor, as this is a unitary operation under summation. Once more applying the chain rule to the partial derivative that is left, we obtain:

$$\begin{aligned}
\frac{d\mathcal{J}^i}{df_x^j} &= \frac{1}{\sum_k f_x^k} \sum_{a,y} \delta_{ai} \frac{\partial \mathcal{J}^i}{\partial \mathcal{I}^a} \frac{\partial \mathcal{I}^a}{\partial \hat{f}^a} (\delta_{aj} - \hat{f}_y^a) \left[\delta_{xy} - \bar{\delta}_{xy} \frac{c_x^j}{\sum_{v \neq x} c_v^j} \frac{\hat{f}_y^j}{\hat{f}_x^j}\right] \\
&= \frac{1}{\sum_k f_x^k} \frac{\partial \mathcal{J}^i}{\partial \mathcal{I}^i} \left\{ \sum_y \frac{\partial \mathcal{I}^i}{\partial \hat{f}_y^i} (\delta_{ij} - \hat{f}_y^i) \left[\delta_{xy} - \bar{\delta}_{xy} \frac{c_x^j}{\sum_{v \neq x} c_v^j} \frac{\hat{f}_y^j}{\hat{f}_x^j}\right] \right\}
\end{aligned} \tag{A.30}$$

We now focus on the summation term over Challenges $y$, and splitting this into the contributions where $y = x$ and $y \neq x$, such that

$$\frac{\partial \mathcal{I}^i}{\partial \hat{f}_x^i} (\delta_{ij} - \hat{f}_x^i) + \sum_{y \neq x} \frac{\partial \mathcal{I}^i}{\partial \hat{f}_y^i} (\delta_{ij} - \hat{f}_y^i) \left(\delta_{xy} - \bar{\delta}_{xy} \frac{c_x^j}{(n-1)\langle c_{y \neq x}^j \rangle} \frac{\hat{f}_y^j}{\hat{f}_x^j}\right) \tag{A.31}$$

where we have used the notation $\langle c_{y \neq x}^j \rangle = \frac{1}{n-1} \sum_{y \neq x} c_y^j$ to denote the average cost of obtaining the challenge factors different from Challenge $x$. Now suppose that Benchmarker $j$

is near $\hat{f}^j$-parity, such that a single factor at Challenge $x$ is moved away from parity, with the rest remaining the same. Additionally, we assume that the rate of change of the unnormalised influence $\mathcal{I}^i$ with respect to the challenge factors is the same for all Challenges different from $x$ near $\hat{f}^j$-parity. Therefore we can simplify the sum in the following way:

$$(\delta_{ij} - \hat{f}^i_x)\frac{\partial \mathcal{I}^i}{\partial \hat{f}^i_x} - \frac{c^j_x}{(n-1)\langle c^j_{y\neq x}\rangle}\frac{\hat{f}^j_{y\neq x}}{\hat{f}^j_x}\frac{\partial \mathcal{I}^i}{\partial \hat{f}^i_{y\neq x}}\sum_{y\neq x}(\delta_{ij} - \hat{f}^i_y) \tag{A.32}$$

Finally, putting it all together and simplifying we arrive at the general derivative result

$$\frac{d\mathcal{J}^i}{df^j_x} = \frac{1}{\sum_k f^k_x}\frac{\hat{f}^j_{y\neq x}}{\langle c^j_{y\neq x}\rangle}\frac{\partial \mathcal{J}^i}{\partial \mathcal{I}^i}\left[(\delta_{ij} - \hat{f}^i_x)\frac{\langle c^j_{y\neq x}\rangle}{\hat{f}^j_{y\neq x}}\frac{\partial \mathcal{I}^i}{\partial \hat{f}^i_x} - (\delta_{ij} - \hat{f}^i_{y\neq x})\frac{c^j_x}{\hat{f}^j_x}\frac{\partial \mathcal{I}^i}{\partial \hat{f}^i_{y\neq x}}\right] \tag{A.33}$$

Under the assumption that here $i = j$, it immediately follows that:

$$\frac{d\mathcal{J}^i}{df^i_x} = \frac{1}{\sum_k f^k_x}\frac{\partial \mathcal{J}^i}{\partial \mathcal{I}^i}\left[(1 - \hat{f}^i_x)\frac{\partial \mathcal{I}^i}{\partial \hat{f}^i_x} - (1 - \hat{f}^i_{y\neq x})\frac{c^i_x}{\hat{f}^i_x}\frac{\hat{f}^i_{y\neq x}}{\langle c^i_{y\neq x}\rangle}\frac{\partial \mathcal{I}^i}{\partial \hat{f}^i_{y\neq x}}\right] \tag{A.34}$$

We can make a further simplification that

$$\frac{c^i_x}{\hat{f}^i_x} = g^i_x\sum_k f^{:} = h^i_x \tag{A.35}$$

for both $y = x$ and $y \neq x$ cases (with the latter of course, only holding if the unit costs for all of the $y \neq x$ Challenges . Interestingly, $h^i_x$ is a measure of the total cost that a Benchmarker $i$ would have to expend in order to obtain a Challenge factor of unity for Challenge $x$. The derivative re-expressed in terms of these quantities is then

$$\frac{d\mathcal{J}^i}{df^i_x} = \frac{1}{\sum_k f^k_x}\frac{\partial \mathcal{J}^i}{\partial \mathcal{I}^i}\left[(1 - \hat{f}^i_x)\frac{\partial \mathcal{I}^i}{\partial \hat{f}^i_x} - (1 - \hat{f}^i_{y\neq x})\frac{h^i_x}{h^i_{y\neq x}}\frac{\partial \mathcal{I}^i}{\partial \hat{f}^i_{y\neq x}}\right] \tag{A.36}$$

The point at which a change in $f^i_x$ switches from having a beneficial to detrimental effect (or vice versa) on the influence is given by the non-trivial stationary point of the derivative $d\mathcal{J}^i/df^i_x$. The trivial stationary point is obtained by computing

$$\frac{\partial \mathcal{J}^i}{\partial \mathcal{I}^j} = \frac{\partial}{\partial \mathcal{I}^j}\left(\frac{\mathcal{I}^i}{\sum_k \mathcal{I}^k}\right) = \frac{\partial \mathcal{I}^i}{\partial \mathcal{I}^j}\frac{1}{\sum_k \mathcal{I}^k} + \mathcal{I}^i\frac{\partial}{\partial \mathcal{I}^j}\left(\frac{1}{\sum_k \mathcal{I}^k}\right) = \frac{\delta_{ij} - \mathcal{J}^i}{\sum_k \mathcal{I}^k} \tag{A.37}$$

The above holds for $\mathcal{J}^i = 1$ for Benchmarker $i = j$ and $\mathcal{J}^i = 0$ when $i \neq j$. To find the non-trivial stationary point, we set to zero the contents of the square brackets, in order to show that

$$\frac{h_x}{h_{y\neq x}} = \frac{1 - \hat{f}_x}{1 - \hat{f}_{y\neq x}}\frac{\partial \mathcal{I}/\partial \hat{f}_x}{\partial \mathcal{I}/\partial \hat{f}_{y\neq x}} \tag{A.38}$$

where we have dropped the $i$ notation denoting Benchmarker $i$. Defining $p_x = h_x/(1 - \hat{f}_x)$ as the cost of one extra unit of share in challenge $x$, we arrive at showing

$$\frac{\partial \log \mathcal{I}/\partial \hat{f}_x}{p_x} = \frac{\partial \log \mathcal{I}/\partial \hat{f}_{y\neq x}}{p_{y\neq x}} \tag{A.39}$$

where in the equation above we replaced the influence $\mathcal{I}$ with it's logarithm. The equations which are listed above give rise to some interesting results pertaining to the protection of a Benchmarkers ability to dominate the market.

### A.3.2 Market domination with uncapped qualification

A Benchmarker $i$'s influence will change with respect to their Challenge factors given by Eqn. A.36

$$\frac{d\mathcal{J}^i}{d\hat{f}_x^i} \propto (1-\hat{f}_x^i)\frac{\partial\mathcal{I}^i}{\partial\hat{f}_x^i} - \frac{h_x}{h_{y\neq x}}(1-\hat{f}_{y\neq x}^i)\frac{\partial\mathcal{I}}{\partial\hat{f}_{y\neq x}^i} \tag{A.40}$$

Simply put, the first term on the right side of the equation above represents the influence gained from the share that one solution buys in Challenge $x$, while the second term represents the loss of influence you give up from the share that the same cost would have bought in challenge $y$. The term $1-\hat{f}_x^i$ represents the *dilution* of the influence gained as the Benchmarker $i$ begins to dominate a certain challenge $x$. As the $\hat{f}_x^i \to 1$, this term falls to 0 and influence gained from additional solutions is diluted down.

The dilution alone provides some protection for the Benchmarking market against domination of a single competitor, even without the addition of an imbalance penalty. Suppose that we switch off the imbalance penalty by setting $k=0$, and assume that the unit costs of all challenges $g_x^i$ is equal. To put this into a numerical context, let's assume that you initially start with a total of 100 qualifiers in each challenge, and everyone else has a total of 900 in each challenge. In this situation it is simple to estimate the challenge factor $\hat{f}_x^i = 100/(100+900) = 0.1$. Now suppose the Benchmarker has access to a larger compute, and is able to generate more qualifying solutions. The results of this numerical experimental are listed in Table. 2

| $f_x$ | $\hat{f}_x$ | Gain $(1-\hat{f}_x)$ | Loss $(h_x/h_y)(1-\hat{f}_{y\neq x})$ | Estimated $d\mathcal{J}/d\hat{f}_x$ |
|---|---|---|---|---|
| 100 | 0.10 | 0.90 | $1.0 \cdot 0.9 = 0.90$ | 0 |
| 300 | 0.25 | 0.75 | $1.2 \cdot 0.9 = 1.08$ | -0.33 |
| 900 | 0.50 | 0.50 | $1.8 \cdot 0.9 = 1.62$ | -1.12 |
| 8100 | 0.90 | 0.10 | $9.0 \cdot 0.9 = 8.10$ | -8.00 |

Table 2: Change in Benchmarker's influence $i$'s based on no imbalance penalty and fixed unit costs.

At parity, the derivate is 0, and every qualifier in Challenge $x$ results in a larger loss of influence than gain. At 90%, each solution loses around 8 times as much influence as it gains. It is interesting to consider the situation, what happens if we no longer assume that the unit costs of obtaining a qualifying solution is a fixed costs. Suppose that in this situation, a Benchmarker has access to some special optimisation for challenge $x$ only[†] which results in the unit costs $g_x$ of Challenge $x$ being substantially smaller than for the other Challenges. To what degree

[†] An example of this could be that a Benchmarker has access to an ASIC.